MGAI2

Ina Vlad **(s4420063)** Neeil Nandal **(s4454537)**
Rutger Berger **(s2498278)**

March 2025

1 Introduction

Over the last decades, the field of game AI has made many memorable achievements. For example, when Google Deepmind developed the AlphaGo algorithm, it became the first Go algorithm to beat a professional human Go player on a full-sized board. In this algorithm, they made use of the monte carlo tree search algorithm (MCTS)Fu [2016]. MCTS is a very popular game tree search algorithm where it explores potential outcomes by running many simulations. It uses the results of those simulations to guide its decisions. This study focuses on applying MCTS to a pacman world, where the goal is to defeat other players by eating dots. We will have a look into what MCTS is by definition and then tweak the algorithm so it fits for our problem. A major part of MCTS relies on how the state evaluation is implemented, so that will be a topic of high interest in this study as well. We will first discuss the theory and then zoom in on our environment and experiments, where we discuss our version of the algorithm.

2 Theory

This section focuses on the theoretical foundation of monte carlo tree search. It will elaborate on the principles of the algorithm which will lead the way to our experimental section, where we define what we do within our experiments.

MCTS

Monte carlo tree search essentially consists of four different phases: selection, expansion, roll-out and backpropagation. The main goal is to traverse the game tree and efficiently generate the optimal tree from the root. During the selection phase, actions are selected within the game tree to create a path to a certain leaf-node. The UCB (upper confidence bounds) selection formula allows us to ensure an exploration during the selection phase:

$$\text{UCB1}(a) = Q(s,a) + c\sqrt{\frac{\ln N_s}{n_a}} \tag{1}$$

Where $Q(s, a)$ is the estimated value of the node after taking action a , N_s is the total number of times the parent node has been visited, n_a how much the child node (which is obtained after taking action a) is visited and c is a constant exploration parameter, which we can alter.

When a leaf is reached, expansion will take place. During this phase, a new action will be taken randomly after which the roll-out phase will take place. The rollout phase, or simulation phase, ensures there will be a number of simulations from the action taken and the reward will be averaged from these roll-outs. Usually, this is performed by taking actions until a termination state is reached, i.e, until the game is either won or lost. However, some games terminate only after a very high number of moves and then a cut-off is set on the roll-out length. After the roll-out phase, tree is updated (backpropagation) with the obtained values, so that the new reward (state value) is incorporated in the tree.

The pseudo code looks as follows:

Algorithm 1 Monte Carlo Tree Search (MCTS) Cycle

```
1: procedure MCTS_Cycle(node)
2:     selected_node ← Selection(node)
3:     reward ← Simulation(selected_node)
4:     Backpropagation(selected_node, reward)
5: end procedure
6: procedure Selection(node)
7:     // Traverse the tree from node to a leaf node
8:     // using a selection strategy (e.g., UCT)
9:     // expands at the end
10:    return selected_leaf_node
11: end procedure
12: procedure Simulation(node)
13:    // Simulate a game from node to a terminal state
14:    // using a rollout policy (e.g., random moves)
15:    return reward
16: end procedure
17: procedure Backpropagation(node, reward)
18:    // Update the statistics of nodes along the path
19: end procedure
```

Heuristics

It is possible to guide the search process a little using heuristics. Heuristics are corner stones of MCTS to increase performance. Within the selection phase, heuristics can help determine which nodes to explore, prioritizing those that seem more promising. It helps to avoid wasting time on some obviously poor traces down the tree. Moreover, during the roll-out phase, heuristics can be used to bias the simulations, making them more realistic and informative. Instead

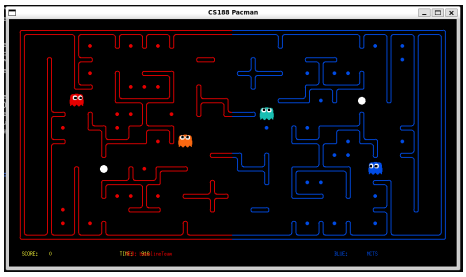

Figure 1: The pacman CTF game.

of purely random actions, heuristics can suggest moves that are more likely to be made by a skilled player. In this study, we will be using a heuristic function which returns a real number to indicate the goodness of the state-action pair.

3 Experiment setup

This section covers the environment, the used configurations of MCTS and the performed experiments.

3.1 The environment

The environment we are using is the pacman CTF game, as described in here. It consists of two teams (red and blue) and the goal is to capture as many dots on the opposing field. When an agent is on the opponent's site, it will become a pacman. When on its own site, it becomes a ghost. It can bring home dots by collecting them and returning to its own site. The game terminates when either all dots are collected or the step limit is reached (a total of 1200 environment steps / 300 per agent). If the environment terminates before all dots are collected, the number of dots collected per team serves as final score. The framework used to incorporate our agent creation logic was created by Sheldon [2021]. An example frame is shown in figure 1

To get our agents perform, we created a team of an offensive and defensive agent. The goal of the offensive agent is to gather as much food as possible, whereas the defensive agent is trying to defend its own food. We believed to let agents win, good performance can be reached by analyzing certain parameters of the state the agent is in, such as the distance to its own and defending food, the distance to invaders, the distance to enemy ghosts or the number of taken home food. In the heuristic section we will further elaborate on the chosen state analysis.

3.2 MCTS configuration

Where the basic structure of MCTS is rather simple, there are many approaches into implementing the algorithm. In this section we will highlight the main possibilities to perform experiments with.

3.2.1 Hyperparameters

One obvious parameter is to decide how many iterations we let our MCTS run for. A higher value increases coverage whereas a lower value increases speed. We will be analyzing this value with ranges from 100-200. Within every iteration, we first perform the selection and expansion phase. During selection, the tree is traversed according to formula 1. For c, a value of 2 is used (a common value, see Świechowski et al. [2023]), this can be an important parameter to increase exploration of the agent. In the roll-out function, the depth-limit is again an important parameter which can have great effect on performance. Setting this value higher will result in a more informative long-term estimate, at the cost of computation time. We experiment with this value to ensure a trade-off between both. As reward function, we use our state evaluation functions as defined in the next section. During roll-out, we can choose to act randomly or act based on heuristics. When chosen on heuristics, the agent will perform actions which optimize the next state value during the roll-out. If not, the agent will perform random actions out of all legal actions.

3.2.2 Heuristics

To guide our search process, we implemented two different heuristic functions. One for the offensive case and one for the defensive case. The offensive behavior takes into account features such as the distance to food dots and power capsules, the closest enemy or scared enemy and the number of food remaining on the board, which is defined as the *successorScore*. Another important parameter we defined is the *desireToHome*. The desire to home calculates how urgent the agent should be going to its home base and is calculated as follows:

$$\text{desireToHome} = \frac{c * \text{numCarrying}}{\text{distanceGhost}} \tag{2}$$

Where c is a constant which is set by manually, a higher c means an earlier desire to home, obviously, whereas if we lower c, it keeps wandering around more. The numCarrying variable refers to how much food the agent is carrying and distanceGhost to how far the agents is from a ghost. The more food it has and the closer the enemy becomes, the higher the agent's desire becomes to bring home the food.

The desireToHome feature is used to combine with another feature, the distanceToHome - which refers to the distance to the spawn point of the pacman. So, the higher the desire to home, the more heavy the distance to home weights in the state evaluation criterium.

The other (offensive) weights were adjusted in a way that makes them penalize the agent when the distance to food is significant and rewards it when the distance to ghosts is large and when there are less food dots left to eat. Final weights are selected according to the ablation study, which will be discussed in a next subsection (or see 2 already). When an agent is considered defensive, a different approach is more convenient. We appreciate the closer the opponent is, the closer our own food is and the less invaders there are. The following formula is used for the defensive state evaluation.

3.2.3 MCTS Improvements

There are numerous improvements possible to implement for our MCTS algorithm. We will be looking at two versions. First, we implement a sufficiency threshold, as suggested by the authors of Świechowski et al. [2023]. In some cases, moves seem very promising at first (even leading to a win), but can be very easy to refute in practice. The idea of the sufficiency threshold is that, when a move seems very good, we almost over-exploit it to gain a better estimate of that node so we then encounter the version where the move actually wasn't so good more often. This is done by calculating the value of c in our UCB formula:

$$c = \begin{cases} c & \text{when all } Q(s,a) \leq \alpha, \\ 0 & \text{when any } Q(s,a) > \alpha. \end{cases}$$

So if there exist any action taken from state s which leads to a significant high value, we ignore the following exploration. The threshold α is now a new hyperparamter, we will set it to -1700 (offensive) and 70 (defensive), which is set by observation.

Another improvement we probed to implement is the adaptive policy improvement. Earlier, MCTS relied on fixed or static policies for its playout and rollout phase. Now, this approach involves dynamically changing the playout policy during the tree search. During implementation, we found this too time-involving and buggy so we were never able to finalize this.

3.3 Experiments

For evaluation, both the offensive and defensive heuristic functions were tested individually with a baseline agent as the other team mate. This was done in order to gain better explainability in terms of features and weights performance for each agent rather than measuring the overall team statistics which includes an interaction effect.

For the first experiment, we perform an ablation study where we start with the baseline implementation. We iteratively add features in our heuristic function and try different weights for these features to see which heuristic values works best. The opponent of the agent is the provided baseline implementation. For each combination, the performance is measured using the ELO score.

Every combination runs 50 times and after each game the ELO score is updated according to the following formula:

$$E_A = \frac{1}{1 + 10^{(R_B - R_A)/400}} \tag{3}$$

$$R'_A = R_A + K(S_A - E_A) \tag{4}$$

Where E_A is the expected score of agent A, R_A the current rating of agent A, R'_A the next rating of agent A, K a constant (4), S_A the score of the game (0 for lose, 0.5 for tie, 1 for win). Initially both agent start with a score of 1000.

After the best heuristic functions are chosen, we continue with these values and we will perform different tournaments. (1) Baseline vs random MCTS (no heuristics used in rollout phase), (2) baseline vs heuristic MCTS (heuristics used in the rollout phase), (3) baseline vs slightly improved MCTS, (4) baseline vs improved MCTS and (5) heuristic reflexive agent vs heuristic MCTS agent. The last one uses the determined state-action evaluation in the action selection but doesn't use the framework of MCTS.

4 Results

Results of the first experiment, where we performed an ablation study starting with the baseline implementation, can be found in Tables 1 and 2. We iteratively added features and tried different values of these features. It is clear to observe that all of the implemented features contained added value to the heuristic function, as the ELO scores increased compared to the baseline for each configuration. For the defensive agent, we see that including a reward for staying near its own food increased robustness of the agent - it never lost since. Looking at ELO scores (including ones of the enemy), the values of -0.001 for the totalFoodDistance and -0.1 for closestFoodDistance suit the best. We also observed that increasing these values too much made the agent loose in seeking enemies, which is its main objective. Therefore we also preferred slightly lower values.

For the offensive agent, we see a high increase in the win rate for all tested features. We continued with a value of 1 for distanceToGhost and a value of 0.5 for distanceScaredGhost. Here as well, we observed during games that increasing these values too much hindered the pacman from its main objective: bringing home food. However, what increased performance most significantly was including a desire to bring home food, where win-rates skyrocketed up to 80%.

After the ablation study and the manual inspection, we found the features according to Table 3 to be most successful and we implemented these accordingly (referred to as 'finetuned heuristic').

numInvaders	onDefense	invaderDistance	totalFoodDistance	closestFoodDistance	ELO score (opponent/own)	win/lose rate
-100	10	-10	-	-	760/910	0.02/0.06
-100	10	-10	-0.1	-	943/988	0.07/0.0
-100	10	-10	-0.01	-	941/988	0.06/0.0
-100	10	-10	**-0.001**	-	952/996	0.07/0.0
-100	10	-10	-	-5	961/996	0.06/0.0
-100	10	-10	-	-1	954/990	0.04/0.0
-100	10	-10	-	**-0.1**	919/985	0.09/0.02

Table 1: The resulting ELO scores with different defensive heuristic values. Distance to food is measured in maze distances, to the food of the agent's team (which it is meant to protect). Experiments are run 5 times for 50 games. For the heuristic agents, without using MCTS, we want to prevent the agents from stopping and reversing the entire time. Therefore, for each configuration, a penalty is given for a stop and reverse action.

closestFoodDistance	successorScore	distanceToGhost	c (distanceToHome)	distanceToCapsule	distanceScaredGhost	ELO score (opponent/own)	win/lose rate
-1	100	-	-	-	-	860/910	0.02/0.03
-1	100	0.5	-	-	-	902/983	0.10/0.00
-1	100	1	-	-	-	843/976	0.20/0.03
-1	100	3	-	-	-	845/985	0.21/0.01
-1	100	-	0.5	-	-	600/979	0.48/0.01
-1	100	-	2	-	-	467/1027	0.82/0.01
-1	100	-	20	-	-	436/1071	0.86/0.08
-1	100	-	-	-0.5	-	817/970	0.17/0.01
-1	100	-	-	-1	-	821/967	0.18/0.01
-1	100	-	-	-5	-	866/987	0.16/0.01
-1	100	-	-	-	0.5	830/973	0.17/0.01
-1	100	-	-	-	1	894/990	0.18/0.02
-1	100	-	-	-	3	858/979	0.17/0.02

Table 2: The resulting ELO scores with different offensive heuristic values. Again, the distance to food and opponents is measured in maze distances, this time to the food of the opponent's team (which it is meant to eat). Here as well, experiments are run 5 times for 50 games.

4.1 MCTS agents

Next, we performed different experiments with MCTS agents. We first performed studies to investigate the optimal hyperparameters for the algorithm (rollout depth and number of simulations), see Tables 5 and 4. For the simulation number, the middle value of 150 shows best behavior. It had a higher average ELO than for 100 iterations and the same win rate, suggesting the agent makes slightly more refined decisions. During 200 iterations, the agent could start prioritizing certain short-term moves that are not beneficial long-term which could explain the lower ELO and win rate. Another reason could be that during the specific set-up of our state evaluation function, agents could prefer actions which aren't necessarily increasing the long-term gains. For example, state features like the distance to home, or the distance to ghosts could be overly important compared to the successor score. Even though the weights minimize this chance, it could still be an important clarification.

For the maximum depth of roll-out trees, a depth of 7 was concluded to be the optimal choice compared to the average ELO and win rate as seen in Table 5. A lower maximum tree, with a depth such as 5, could prioritize short-term rewards and act too greedy while ignoring upcoming danger. In contrast, a higher tree

Pacman Features	Ghost Features
isPacman (w=-5)	numInvaders (w=-100)
distanceToFood (w=-1)	onDefense (w=10)
successorScore (w=-100)	invaderDistance (w=-10)
distanceToHome (2) * (w=-1)	closesFoodDistance (w=-0.1)
invaderDistance (defending) (w=-1)	totalFoodDistance (w=-0.001)
distanceCapsule (w=-1)	-
distanceGhost(w=0.5)	-

Table 3: Pacman and Ghost Feature Weights

Hyperparameter	Average ELO	Enemy ELO	Win Rate (%)
simulations=100	1317.27	1038.04	58.00
simulations=150	1434.14	1016.34	58.00
simulations=200	1292.3	995.9	50.00

Table 4: Results for different nr. of simulations in MCTS (using random rollouts). The opposing agent used is the provided baseline agent.

depth of 10 is outperformed by the lower depth value and perhaps predicts too far ahead, ignoring dynamic changes that lead to dangerous situations. Therefore, a tree depth of 7 balances both planning in advance and reacting to dynamic changes which explains the higher ELO and win rate.

Next, we performed a tournament with the MCTS agents. Results of these can be found in 6. For the first two versions, the heuristic in state evaluation excluded the desireToHome feature and had the features determined as by the ablation study. Using these heuristics in the search process appears to be slightly successful. After running for 50 games, the resulting ELO score is 1317.27 for our team as seen in Table 6. The win rate is 36.54%, which is explained by the high number of tie games as observed in the tournament record.

Consequently, an experiment was also conducted to assess the performance when our agents use MCTS with random rollouts (no heuristic used in the search process). This configuration outperformed the heuristic version of MCTS in terms of win rate and obtained ELO, as observable as well in Table 6. An explanation for this would be that having random rollouts, the agent gets to explore more of the environment and avoids premature commitment to earlier determined strategies, which apparently may guide the behavior of the agents to a lower win rate. The random rollout gains more information of the environment compared to the heuristic MCTS, allthough the difference is still quite little.

Most important results from 6 show the astonishing effect of the manually optimized heuristic (including desireToHome) and the limited effect of using MCTS. Where we see that the finetuned heuristic has a win-rate of 84% against a baseline agent, we also see in the fourth line that the same MCTS agent has trouble in beating an agent which solely used the heuristics (which evaluates

Hyperparameter	Average ELO	Enemy ELO	Win Rate
depth=5	1071.77	972.3	36.84
depth = 7	1317.27	1038.04	58.00
depth = 10	1221.10	1035.7	44.00

Table 5: Results for different max tree depths in MCTS (using random rollouts). The opposing agent used is the provided baseline agent.

Agent	Opponent	Average ELO	Enemy ELO	Agent Win Rate (%)
MCTS + random rollouts	baseline	1371.21	1010.01	58.00
MCTS + untested heuristic	baseline	1317.27	1038.04	36.54
MCTS + finetuned heuristic	baseline	1569.42	940.02	84.00
MCTS + heuristic	reflexive heuristic	1020.37	1170.36	32.00
MCTS + improvements	baseline	1460.00	980.51	88.00

Table 6: Final evaluation scores for each agent configuration of MCTS, using the best found earlier hyperparameters. The opponents in these experiments are the baseline agents.

only the possible actions from that state).

5 Discussion

We've looked at many different hyperparameters and heuristic evaluation functions. Our most significant result was the impact of the desireToHome function in the offensive agent and the distanceToFood for the defending agent. Where we first determined all values by ablation study using heuristic agents, we saw that these values were not exactly transferable to using with our MCTS agents and required some manual adaptation by observation. The planning nature of MCTS may be influential to the performance compared to the reflexive heuristic agents.

Moreover, where it is commonly expected that an increase in number of iterations in MCTS and an increase in the roll-out depth would increase performance linearly, this was also not the case. We saw that the win-rates and ELO scores were the highest for mid-range values. On top of that, the random roll-out MCTS outperformed our initial MCTS agent (with no fine-tuned heuristic). The design of our heuristic function is rather restrictive to the MCTS agent and could also hinder the MCTS agent in reaching ultimate behaviour.

Our heuristics were actually designed for short-term goals: gain food, stay away from ghosts, bring home food if needed. However, MCTS typically works well if it could find its own path to long-term objectives (win a game). This was actually due to time restrictions hard to implement as typically games would run for 1200 steps. One simulation using 1200 steps already takes up to one minute and is typically not very informative.

After having our heuristics fine-tuned for MCTS, we saw very good behaviour

for the MCTS against the baseline agents: a win-rate of 98% and ELO-scores of over 1600. Letting it play against a reflexive agent with the same heuristics showed however that the heuristics were the power of the algorithm - and not the MCTS itself.

5.1 Future Work

Future improvements in the configuration of the agents could include the use of specific tactics that get triggered based on thresholds, rather than only using a heuristic for the MCTS rollout. Ikehata and Ito [2011] used a survival tactic that evaluates the survival rates of different paths in search of safe spots, a feeding tactic that prioritizes food rather than safety and a pursuit tactic activated when the enemies are in scared mode. They also build a danger level map resembling a heat map that is used by the agent to guide its path selection. These additional strategies could be more robust in enhancing the performance of the agent as fine-tuning weights is a more sensitive process.

Another approach that could refine the ghost agent is creating multiple ghost types with different behavior (chaser, ambusher, scatter, tactical) as in the work of Yanovsky [2024]. An adaptation of this method in our environment that only allows at most two ghosts, would be to test and analyze which two ghost models perform the best together. Yanovsky [2024] also analyzed the win rate and average score when increasing the number of MCTS simulations from 50 up to 300 on both optimized and non-optimized mazes. Therefore, testing the performance of our agents on different maze layouts and sizes could provide more insight regarding the robustness of our configuration and also put the agents in new scenarios where they could perform suboptimally.

References

Michael C Fu. Alphago and monte carlo tree search: the simulation optimization perspective. In *2016 Winter Simulation Conference (WSC)*, pages 659–670. IEEE, 2016.

Nozomu Ikehata and Takeshi Ito. *Monte-Carlo tree search in Ms. Pac-Man.* IEEE, 10 2011. doi: 10.1109/CIG.2011.6031987.

Christian Sheldon. Python3 version of uc berkeley's cs 188 pacman capture the flag project. `https://github.com/cshelton/pacman-ctf`, 2021. Computer software.

Maciej Świechowski, Konrad Godlewski, Bartosz Sawicki, and Jacek Mańdziuk. Monte carlo tree search: A review of recent modifications and applications. *Artificial Intelligence Review*, 56(3):2497–2562, 2023.

Vladimir Yanovsky. Exploring the limits of mcts in pac-man: Maze size, simulations, and performance. *Herald of Khmelnytskyi National Univer-*

sity, 341(, 2024. doi: 10.31891/2307-5732-2024-341-5-52. URL https://heraldts.khmnu.edu.ua/index.php/heraldts/article/view/671.